



**KLE Technological
University** | Creating Value,
Leveraging Knowledge

Dr. M. S. Sheshgiri Campus, Belagavi

LINEAR ALGEBRA LAB

Course Code: 25EMAB103

PROJECT REPORT ON

GAUSSIAN ELIMINATION SOLVER WITH STEP-BY-STEP OUTPUT

Submitted for the completion of credit course of

BACHELOR OF ENGINEERING

Submitted By

1. **KARTHIK HIRENARTI** 02FE25BCS044
2. **VIKAS BASTAWAD** 02FE25BCS045
3. **GANGAMMA NAGANUR** 02FE25BCS043
4. **SHREYA GAIKWAD** 02FE25BCS046

Under the Guidance of

Dr. Shoba Patil

Prof. Veena B



**KLE Technological
University** | Creating Value,
Leveraging Knowledge

Dr. M. S. Sheshgiri Campus, Belagavi

Dr. M. S. Sheshgiri Campus, Belagavi

DEPT OF MATHEMATICS

CERTIFICATE

This is to certify that the course project titled “**GAUSSIAN ELIMINATION SOLVER WITH STEP-BY-STEP OUTPUT**” has been carried out by the students Karthik Hirennarti (02FE25BCS044), Vikas Bastawad (02FE25BCS045), Gangamma Naganur (02FE25BCS043), and Shreya Gaikwad (02FE25BCS046) as part of the **Linear Algebra Lab** course (25EMAB103) during the 2nd Semester of the B.E. program for the academic year 2025-26. The project report fulfils the requirements prescribed by KLE Technological University.

Sl.No	NAME	SRN	STUDENT-SIGN
1.	Karthik Hirennarti	02FE25BCS044	
2.	Vikas Bastawad	02FE25BCS045	
3.	Gangamma Naganur	02FE25BCS043	
4.	Shreya Gaikwad	02FE25BCS046	

Dr. Shoba Patil
Guide

Prof. Veena Pattar
Guide

Contents

1	Abstract	3
2	Introduction	3
	2.1 Mathematical Concept	3
	2.2 Project Scope and Integration	3
	2.3 Real-World Applications	3
3	Problem Statement	4
	3.1 Challenges of Manual Calculations	4
	3.2 The Black-Box Computational Gap	4
4	Objectives	4
5	Background Theory	5
	5.1 Augmented Matrix	5
	5.2 Elementary Row Operations	6
	5.3 Row Echelon Form and Back Substitution	6
6	Workflow / Flowchart	6
7	Algorithm	7
8	Pseudocode	8
9	Python Implementation	9
	Python Source Code	9
10	Results and Outputs	11
	10.1 Sample Execution Details	11
	10.2 Back Substitution Solver Steps	11
11	Applications	12
12	Conclusion	12
13	References	12

1 Abstract

Solving systems of linear equations is a fundamental prerequisite in scientific computing, mathematical modeling, and engineering applications. This project report presents a custom-developed, interactive Python-based **Gaussian Elimination Solver** that provides detailed, step-by-step row reduction visualization.

While typical software libraries output only the final solutions, this tool visualizes each intermediate augmented matrix state as row operations are executed. Utilizing Python's robust **NumPy** framework, the system captures user-defined matrices, constructs augmented representations, executes forward elimination to obtain the Row Echelon Form, handles zero-pivot cases, and computes precise values via back substitution. This solver bridges the gap between manual calculation errors and black-box computations, serving as an advanced educational and analytical tool.

2 Introduction

Linear algebra serves as the mathematical foundation for modern computer science, numerical simulation, and data structures. Among various numerical algorithms, **Gaussian Elimination** is one of the most reliable direct methods for solving linear equations of the form $AX = B$.

2.1 Mathematical Concept

Systems of linear equations arise naturally when modeling networks, balancing chemistry equations, or performing physical transformations. Manual execution using row operations is highly error-prone, especially for systems where dimensions $n \geq 3$. Systematic computer algorithms solve these systems rapidly by organizing coefficients into matrices.

2.2 Project Scope and Integration

This project implements Gaussian Elimination in Python using the NumPy array manipulation framework. The software visualizes step-by-step state modifications of the augmented matrix. This visual framework makes the matrix transformations clear and assists students in grasping row operations.

2.3 Real-World Applications

Advanced matrix computing models are widely used in multiple disciplines:

1. **Artificial Intelligence & Machine Learning:** Optimizing weights in linear regression and feedforward networks.

2. **Data Science:** Supporting dimensionality reduction methods like Principal Component Analysis (PCA).
3. **Engineering Calculations:** Finding numerical solutions for electrical mesh/nodal analysis and civil structures.
4. **Circuit Analysis:** Formulating simultaneous nodal equations based on Kirchhoff's Current Law.
5. **Computer Graphics:** Applying transformations, scales, and perspective projections using 4×4 matrices.

3 Problem Statement

Develop a Python-based Gaussian Elimination Solver that performs and visualizes each step of row reduction on user-defined systems of linear equations.

3.1 Challenges of Manual Calculations

Manually solving large matrices is slow and prone to errors. Fractional numbers often introduce rounding and calculation errors that propagate throughout the elimination process, leading to incorrect final solutions.

3.2 The Black-Box Computational Gap

While programming languages provide integrated routines (such as `numpy.linalg.solve`) to solve linear systems instantly, they hide the operational details. There is a lack of educational utilities that display the step-by-step row operations, making it harder for learners to verify their hand-calculated steps.

4 Objectives

The key objectives of this laboratory project are:

- **Implement Row Reduction:** Write a clean, modular Python codebase to construct and manipulate augmented matrix structures without using high-level black-box solvers.
- **Visualize Steps:** Provide precise visual printouts of the augmented matrix after every division or subtraction operation.
- **Handle User Input:** Design an interactive terminal interface supporting custom dimensions and variable coefficients.

- **Apply Mathematical Logic:** Formulate robust pivoting and division routines using the standard Python NumPy library.
- **Integrate Core Linear Algebra Concepts:** Implement forward elimination, row-echelon conversion, and back substitution.

5 Background Theory

A system of n linear equations with n variables can be written compactly as:

$$AX = B \quad (1)$$

where:

- A is the $n \times n$ coefficient matrix.
- X is the $n \times 1$ variable vector.
- B is the $n \times 1$ constant vector.

5.1 Augmented Matrix

By appending the vector B onto the right-hand column of the coefficient matrix A , we form the Augmented Matrix $[A \mid B]$:

$$[A \mid B] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right] \quad (2)$$

For a sample 3×3 system:

$$\begin{aligned} 2x_1 + x_2 - x_3 &= 8 \\ -3x_1 - x_2 + 2x_3 &= -11 \\ -2x_1 + x_2 + 2x_3 &= -3 \end{aligned}$$

The augmented matrix is represented as:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right] \quad (3)$$

5.2 Elementary Row Operations

Gaussian elimination operates on the augmented matrix using three valid operations:

1. **Row Swapping** ($R_i \leftrightarrow R_j$): Exchanging the positions of two rows.
2. **Row Scaling** ($R_i \leftarrow kR_i$): Scaling all entries in a row by a non-zero constant k .
3. **Row Addition/Subtraction** ($R_i \leftarrow R_i + kR_j$): Adding or subtracting a multiple of one row to another.

5.3 Row Echelon Form and Back Substitution

The elimination process systematically yields a matrix in Row Echelon Form (REF), where all elements below the diagonal pivots are reduced to zero. Finally, ****Back Substitution**** is applied starting from the last equation to solve for each variable:

$$x_i = \text{aug}[i][n] - \sum_{j=i+1}^{n-1} \text{aug}[i][j] \times x_j \quad (4)$$

6 Workflow / Flowchart

The diagram below outlines the logical sequence of operations in the solver.

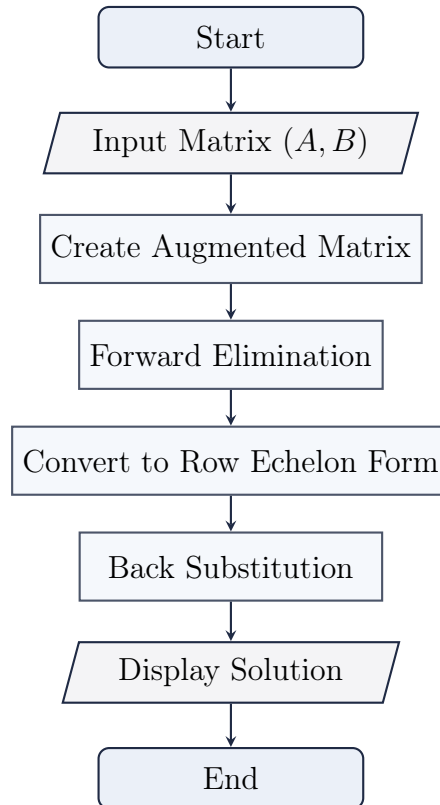


Figure 1: Gaussian Elimination Solver Flowchart

7 Algorithm

The step-by-step algorithm to perform row reduction is:

1. Read the system size n .
2. Prompt the user for the coefficients of the $n \times n$ matrix A and the constants in the vector B .
3. Form the augmented matrix $\text{aug} = [A \mid B]$ of dimensions $n \times (n + 1)$.
4. Loop through each column i from 0 to $n - 1$:
 - (a) If the pivot entry $\text{aug}[i][i]$ is zero, report a zero-pivot exception and terminate.
 - (b) Divide row i by the pivot $\text{aug}[i][i]$ to set the diagonal element to 1. Display the resulting matrix.
 - (c) Eliminate all elements below the pivot by performing row operations:

$$\text{row}_j \leftarrow \text{row}_j - (\text{aug}[j][i] \times \text{row}_i) \quad \text{for } j > i$$

- (d) Output the updated augmented matrix at each step.
5. Initialize a solution vector x of size n .
6. Perform back substitution:

$$x_i = \text{aug}[i][n] - \sum_{j=i+1}^{n-1} \text{aug}[i][j] \times x_j \quad \text{from } i = n - 1 \text{ down to } 0$$

7. Output the final solution values.

8 Pseudocode

Listing 1: Pseudocode for Visual Row Reduction

```
1 Procedure GaussianElimination(A, B):
2     n = length(B)
3     aug = concatenate_columns(A, B)
4     Print "Initial Augmented Matrix"
5
6     For i from 0 to n-1:
7         if aug[i][i] == 0:
8             Print "Zero pivot encountered!"
9             Return
10
11     pivot = aug[i][i]
12     aug[i] = aug[i] / pivot
13     Print "Normalized Pivot Row"
14
15     For j from i+1 to n-1:
16         factor = aug[j][i]
17         aug[j] = aug[j] - factor * aug[i]
18         Print "Eliminated column element below pivot"
19
20     x = array_of_size(n, fill=0)
21     For i from n-1 down to 0:
22         x[i] = aug[i][n]
23         For j from i+1 to n-1:
24             x[i] = x[i] - aug[i][j] * x[j]
25
26     Print "Final solutions:", x
27 End Procedure
```

9 Python Implementation

Python Source Code

```
1 import numpy as np
2
3 def gaussian_elimination(A, B):
4
5     n = len(B)
6
7     aug = np.hstack((A, B.reshape(-1,1)))
8
9     print("\nInitial Augmented Matrix:\n")
10    print(aug)
11
12    for i in range(n):
13
14        if aug[i][i] == 0:
15            print("Zero pivot encountered!")
16            return
17
18        pivot = aug[i][i]
19        aug[i] = aug[i] / pivot
20
21        print(f"\nStep {i+1}: Make Pivot Element 1")
22        print(aug)
23
24        for j in range(i+1, n):
25
26            factor = aug[j][i]
27            aug[j] = aug[j] - factor * aug[i]
28
29            print(f"\nEliminating element in Row {j+1}")
30            print(aug)
31
32    x = np.zeros(n)
33
34    for i in range(n-1, -1, -1):
35
36        x[i] = aug[i][n]
37
38        for j in range(i+1, n):
39            x[i] = x[i] - aug[i][j] * x[j]
40
41    print("\nFinal Row Echelon Form:\n")
42    print(aug)
43
44    print("\nSolution:")
```

```
45     for i in range(n):
46         print(f"x{i+1} = {x[i]}")
47
48     # Interactive matrix inputs
49     n = int(input("Enter number of variables: "))
50
51     print("\nEnter Coefficient Matrix:")
52
53     A = []
54
55     for i in range(n):
56         row = list(map(float, input(f"Enter row {i+1}: ").split()))
57         A.append(row)
58
59     A = np.array(A)
60
61     print("\nEnter Constant Matrix:")
62
63     B = []
64
65     for i in range(n):
66         value = float(input(f"Enter constant {i+1}: "))
67         B.append(value)
68
69     B = np.array(B)
70
71     gaussian_elimination(A, B)
```

10 Results and Outputs

10.1 Sample Execution Details

Using the coefficient matrix A and constant vector B from our background section:

$$A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}$$

The step-by-step reduction is tracked in the table below.

Table 1: Iterative Augmented Matrix States

Operation Step	Augmented Matrix State
Initial Matrix	$\left[\begin{array}{ccc c} 2.0 & 1.0 & -1.0 & 8.0 \\ -3.0 & -1.0 & 2.0 & -11.0 \\ -2.0 & 1.0 & 2.0 & -3.0 \end{array} \right]$
Row 1 Normalized	$\left[\begin{array}{ccc c} 1.0 & 0.5 & -0.5 & 4.0 \\ -3.0 & -1.0 & 2.0 & -11.0 \\ -2.0 & 1.0 & 2.0 & -3.0 \end{array} \right]$
Row 2 Eliminated	$\left[\begin{array}{ccc c} 1.0 & 0.5 & -0.5 & 4.0 \\ 0.0 & 0.5 & 0.5 & 1.0 \\ -2.0 & 1.0 & 2.0 & -3.0 \end{array} \right]$
Row 3 Eliminated	$\left[\begin{array}{ccc c} 1.0 & 0.5 & -0.5 & 4.0 \\ 0.0 & 0.5 & 0.5 & 1.0 \\ 0.0 & 2.0 & 1.0 & 5.0 \end{array} \right]$
Row 2 Normalized	$\left[\begin{array}{ccc c} 1.0 & 0.5 & -0.5 & 4.0 \\ 0.0 & 1.0 & 1.0 & 2.0 \\ 0.0 & 2.0 & 1.0 & 5.0 \end{array} \right]$
Row 3 Normalized (REF)	$\left[\begin{array}{ccc c} 1.0 & 0.5 & -0.5 & 4.0 \\ 0.0 & 1.0 & 1.0 & 2.0 \\ 0.0 & 0.0 & 1.0 & -1.0 \end{array} \right]$

10.2 Back Substitution Solver Steps

1. Row 3 yields: $x_3 = -1.0$
2. Row 2 yields: $x_2 = 2.0 - (1.0 \times x_3) = 2.0 - (-1.0) = 3.0$
3. Row 1 yields: $x_1 = 4.0 - (0.5 \times x_2) - (-0.5 \times x_3) = 4.0 - 1.5 + 0.5(-1.0) = 2.0$

Thus, the final solutions are $x_1 = 2.0$, $x_2 = 3.0$, and $x_3 = -1.0$.

11 Applications

Developing this solver underscores a series of practical implications in high-tech fields:

- **Artificial Intelligence:** Powering linear solvers for linear regression, neural network training forward loops, and support vector machines.
- **Machine Learning (ML):** Solving normal equations directly for least-squares regression estimation.
- **Image Processing:** Calculating transformation matrices for resizing, shearing, rotating, and morphing discrete pixel representations.
- **Data Science:** Supporting computational pipelines that compute eigenvalues, eigenvectors, and singular value decompositions.
- **Engineering:** Analyzing multi-loop electrical circuits and civil structural truss frames.

12 Conclusion

This project successfully designed and implemented a pedagogical **Gaussian Elimination Solver** in Python. By visualising row-scaling and row-elimination phases iteratively, the system renders matrix algorithms transparent and easily verifiable.

Through this laboratory project, our team consolidated critical linear algebra competencies, thoroughly digested augmented matrix dynamics, mastered elementary operations, and sharpened programming practices using the **NumPy** framework. This highlights the indispensable utility of matrix techniques in forming computational solvers for complex physical and digital challenges.

13 References

1. Class Notes — Linear Algebra (25EMAB103)
2. Linear Algebra Lab Python Notes
3. ChatGPT and Claude (Anthropic) — For AI-assisted code debugging, optimization, and structuring
4. GeeksforGeeks — Linear Algebra tutorials and concepts on Gaussian elimination and pivoting operations